GREEN POLICY INTELLIGENCE ENGINE (GPIE)

So this right here is my Development Log. I maintain a log like this for pretty much every research project I work on. Initially, I jst started doing this for my own personal benefit—mostly so I wouldn't lose my mind trying to remember where I left things off the night before, or wht exact dataset was sitting on my disk.

But for this project, I chose to write it down as a proper, ongoing story. Wanted to narrate the actual execution of the project step by step, just in case anyone else wants to see how the whole thing came together. I hve mentioned all of my core reasonings, my sudden dataset changes, nd even those annoying coding bugs and debugging sessions where things completely broke down before I fixed them. It is basically the raw, behind the scenes diary of how this project actually got built.

Index

1. Entry 1
2. Entry 2
3. Entry 3
4. Entry 4
5. Entry 5
6. Entry 6
7. Entry 7
8. Entry 8
9. Entry 9
10. Entry 10
11. Entry 11
12. Entry 12
13. Entry 13
14. Entry 14
15. Entry 15
16. Entry 16
17. Entry 17
18. Entry 18
19. Entry 19
20. Entry 20
21. Entry 21
22. Entry 22
23. Entry 23
24. Entry 24
25. Entry 25
26. Entry 26
27. Entry 27
28. Entry 28
29. Entry 29
30. Entry 30
31. Entry 31

32. Entry 32
33. Entry 33
34. Entry 34
35. Entry 35
36. Entry 36
37. Entry 37

Entry 1

SCRAPER BUILD LOG — EUROPEAN GREEN DEAL LAWS

I needed the actual text records of the European Green Deal laws before messing with any satellite maps. So I did not want a vague reference list (so I chose to build an automated scraper from scratch against EUR Lex because it is the official European Union legal repository). I started very small by testing a single simple web request with the `requests` library nd BeautifulSoup. Then I kept changing the code until it could crawl through every single search result page on its own.

My build process went in slow steps. First, I jst grabbed a single law title n its web link off one page. Next, I updated the loop to crawl full pages so I could pull down the title, URL, nd official legal status for absolutely every record on screen. I then forced the script to click inside every individual link to grab the raw document text. After tht, I added a few metadata steps: I pulled out the policy type nd year straight from the top document heading, calculated the word count and character length using standard string tools, nd chopped the CELEX ID number directly out of the URL string. So I also grabbed a tiny summary from the very first line of the header text n wrote a basic keyword tracker to slap tags on climate relevant policies.

My code wrote everything out into both `documents.json` nd `documents.csv` files. So I immediately ran a quick first pass exploratory data check inside Pandas (counting how many files I had per policy type and checking average lengths by year with some simple bar charts). This was just a quick sanity check to ensure my raw text dataset was complete and matched internally before I wasted real coding time on the heavy satellite tracking script.

CLEANING UP THE SCRAPER CODE

Once the full scraping code finally worked from start to end, I went back and cleaned up the whole mess. Chose to chop up the monolithic code block nd group the steps (metadata extraction, tag building, status reading, nd saving) into their own distinct custom functions (`extract_metadata()`, `generate_tags()`, `export_json()`, `export_csv()`, nd `load_dataframe()`). Then, I wrapped all of tht inside a single high level `scrape_policies()` function n wired the whole workflow through a clean `main()` entry block instead of letting a loose, line by line script run wild. The output files did not change nd the data numbers stayed exactly the same. But I ended up with a neat setup that I could actually edit later without worrying about breaking everything with one wrong click. This code cleanup step mattered to me because I was planning to copy this exact same modular and crash proof coding setup for my heavy satellite data script next. I just wanted to verify tht my custom pipeline setup worked smoothly on this simpler text data first.

Entry 2

SPECIFYING THE SATELLITE SETUP

I finished the legal scraping task and immediately locked down my data collection rules before downloading a single satellite picture. Chose the European Union for my study zone. My time window runs from January 2019 to December 2024 (which gives me a solid pre Green Deal baseline plus several years of actual policy data to look at). I kept my CRS strictly on WGS84 for everything because I do not want to reproject layers unless some future calculation explicitly forces me to. So I made a final list of the layers my code will need: Sentinel-5P NO2, Sentinel-2 NDVI, ESA WorldCover, Copernicus DEM, ERA5 weather stats, WorldPop maps, Eurostat economic numbers, n NUTS boundary shapes.

For the NO2 data, I wrote down a very specific step by step pipeline. So I decided to pull down RPRO (reprocessed) Level-2 items instead of standard NRTI or OFFL files because I am tracking multi year trends rather than doing real time daily monitoring. I set my processing mesh to a 0.05° grid size. Also used the official ESA rule to drop out bad data by setting my quality gate at `qa_value ≥ 0.75`. If pixels are completely missing, I fill them with NaN rather than guessing values with an interpolation tool. To stop big orbital data folders from breaking my hard drive storage, I chose a monthly download then process then delete loop. So I did not guess any of these setup rules. Every choice has a clear engineering reason (like using simple rectangular bounding boxes for my search queries because full polygon shapes always cause the connection to time out). I put this plan down in black nd white now so I do not start arguing with myself mid build later.

I finally got the login system working against the Copernicus Data Space Ecosystem. Chose to use the OAuth2 password grant setup and wrote a simple, reusable helper function to handle the token generation automatically. Next, I made my very first successful logged in web request against their actual product catalog. This let me verify tht my code could pull down real Sentinel-5P item metadata (like the ID, filename, dates, nd geographic footprints) before I sat down to write any complex file download scripts. So I then filtered the system catalog down to only grab the specific NO2 items that overlap my European bounding box coordinates and fit inside my chosen time window. That query worked cleanly on my very first attempt. 😀

Entry 3

So I built out the actual download nd configuration layer. Chose to put every single study parameter (like the bounding box, dates, API endpoints, and directory paths) inside a centralized `config.py` file so nothing was hardcoded twice in different places. Next, I wrote a download utility script with built in file existence nd size verification checks. This means my re runs automatically skip anything that already downloaded correctly, nd I added automatic directory creation so the pipeline never fails jst because a folder is missing. This entire month wise lifecycle (discover, differential download, preprocess, verify, n clear raw files) locked in as the standard shape tht every dataset in this project would follow.

I got HARP (the scientific processing library for Level-2 to Level-3 conversion) working in my conda environment after sorting out a messy dependency chain. Then, I validated it against one real Sentinel-5P product file..... I chose to verify the variable inventory, pulled out `tropospheric_NO2_column_number_density` along with its exact lat/lon coordinates using HARP's `keep()` operation, and exported the whole thing to a CSV file to check if the extraction had silently dropped or corrupted anything. It hadn't. My data showed that the full spatial coverage was perfectly preserved, the raw floating point values stayed intact, nd absolutely no filtering was applied yet. That single product validation became the exact template that my full batch pipeline would run at scale.

Entry 4

So I went through the pipeline hardening steps that a lot of projects skip until something breaks, starting with the download layer. I rebuilt it completely so one failed file does not take down the whole batch run. I chose to set up three retries with backoff per file, post download size verification against the API's own `ContentLength` metadata, nd ran a live test that confirmed both the skip already verified behavior nd the retry behavior under a manual network interruption. Wrapped the HARP preprocessing step into a proper `preprocess_file()` function with the quality filter and spatial binning baked into the exact same operation chain. It returns `None` on failure instead of raising an error so a bad file does not crash the whole run.

I built out the temporal side of the code too. So I wrote `generate_monthly_ranges()` inside `date_utils.py` to produce clean month boundaries for any year range I pass to it. Then, I chose to rebuild `run_pipeline.py` as a real month wise orchestrator script with structured logging routed straight to a local `logs/` directory. I set up fault isolation at both the month level (meaning a failed download jst logs the error n moves on to the next chunk) nd the individual file level. If a file fails to preprocess, my code keeps the raw data around for a retry later instead of deleting it immediately. Tested this whole cycle on a single month first before trusting it against the full 72 month data run.

Entry 5

Finished hardening the NO2 pipeline nd picked up three more datasets in the same session. The Copernicus DEM (30m elevation layer) turned out to sit in a public, no auth S3 storage bucket. So I wrote a quick tile name generator script that matches the official naming system perfectly..... Then, I added a remote size check using HTTP HEAD requests before downloading anything, and I verified tht a single live test tile (Netherlands, 10.2 MB) landed correctly on my disk before committing to the full 1,500 to 3,000 tile check across the whole European zone.

The NUTS administrative boundaries data came straight from Eurostat's GISCO API as a single GeoJSON file. It was much simpler because I did not need any complex per tile download code. The raw file included absolutely every NUTS entity on record (like EFTA members, candidates, n non members). So, I built a strict EU-27 ISO2 n ISO3 mapping rule to clean it up nd filter it down to exactly 27 countries which included a manual fix for Eurostat's non standard "EL" country code for Greece. That filtered country list became the reusable roster tht every other per country script in this project would drive off later.

The WorldPop population data turned out to be the only dataset I couldn't fully wrap up during this coding session. The REST API's official "Global 1" dataset only goes from 2000 to 2020. Huh? So, I chose to limit my script execution explicitly to the 2019–2020 window rather than quietly leaving a blind gap or guessing at some sketchy access path for the 2021–2024 years (which actually sit in WorldPop's newer "Global 2" dataset over on the HDX platform, a completely different system I haven't messed with yet). So I wired the download loop to iterate over the exact same EU-27 country list from my NUTS boundary script. Also added a bit of code for FTP to HTTPS URL normalization because WorldPop's internal file links still use legacy FTP pathways.

So I left the 2021–2024 population data as an open task on my to do list instead of trying to fake a fix. By the time I shut down my computer, the state of the codebase was very clear: my NO2 script was hardened but not yet run at full multi year scale, the DEM data stood verified at just one test tile, the NUTS layer was completely done, the population download block stayed partial by choice, nd I hadn't even started writing the causal inference or economic data modules.

Entry 6

So yeah, I tried to get the NDVI data script moving n hit a real dead end. Ahh, tht's a tough one. Raw Sentinel-2 image tiles are jst way too large for full EU six year coverage to ever fit on my local hard drive, and the obvious cloud storage fix (Sentinel Hub's API) required a separate OAuth setup tht I had not built or tested yet. So, I went looking for a pre computed NDVI dataset instead. I chose this path because NDVI is just a standard math formula (`(NIR - Red) / (NIR + Red)`) anyway, meaning I lose absolutely zero methodological control compared to computing those satellite bands from scratch myself.

Landed on the Copernicus Global Land Service NDVI product (which gives 300m resolution across 10 daily cycles via their Version 3 files) on the main CLMS portal. Registered an account, generated a service account key (which uses a JWT based credential layout tht looks structurally different from every other login pattern in this project so far), nd started writing code against CLMS's official machine to machine download API. So I also found—nd intentionally avoided—an alternate pathway to grab the exact same data using CDSE's newer openEO API. Skipped tht route on purpose because tht specific pathway fails silently on a bad web request rather than throwing a loud error code (which is exactly the nightmare thing you do not want to discover halfway through a huge batch run). Chose to leave this task as a documented next step on my roadmap rather than trying to force it through in a rush: I need to register, grab the token, read the actual machine to machine API docs fully, test on just one file, nd then scale up.

Entry 7

The Eurostat regional GDP data (`nama_10r_2gdp`) turned out to be the absolute simplest thing to download in this whole project. Thats kinda relaxing. One single unauthenticated REST web call returns every NUTS region code together in one single response string. I did not need any per country loop loops, n I did not even bother adding any complex retry code logic given how low risk a tiny JSON file response from a super stable government API server actually is. So I chose to double check the exact dataset string code against multiple independent documentation sources instead of jst trusting my own memory. Filtered the data down to my specific study years right on their server using the API's built in date parameters, nd then I wrote a very basic differential download file check so re running the script skips fetching the file if it is already sitting on my drive.

Entry 8

The ESA WorldCover map layer (which gives 10m global land cover data broken into 11 distinct classes) lives inside a public S3 bucket. It follows the exact same download pattern as my DEM script. I picked version v200 (2021) over the older v100 (2020) file because it gives a much more accurate n up to date classification layout. Made a strict note for later: comparing these two versions directly would mix up real land use shifts with the algorithmic model differences between the two software versions so I chose to use v200 completely on its own as my project baseline instead of faking a v100 versus v200 change tracking test. I built a custom tile name generator script to match WorldCover's native 3° grid setup (which is different from the DEM tool's 1° grid sizing). Next, I modeled this new data collection code closely on my existing DEM script. So I left the actual single tile download verification test as my next concrete task before committing to a full scale file download.

Entry 9

So I had to change my code style completely for the ERA5 weather data (like 2m temperature and rain maps). The CDS API works on a slow job queue where you submit a request, wait around, n then download the file. The main client library code handles tht waiting part internally, so I did not need to write my own looping check script. I had to register a completely fresh ECMWF account because the whole Climate Data Store moved to a new system recently, nd my old login codes would not work anymore. So I also had to click the accept button for the dataset license inside my web browser because the API does not let you automate tht step.

My `download_era5_year()` script pulled all 12 months of data in one single web call. I kept the map view locked to the exact same European bounding box coordinates, and a live 2019 test file came back completely clean before I started the full six year batch run. 😃

Entry 10

So I tried to turn the 233 raw WorldCover map tiles into actual usable stats..... It took me three failed coding attempts to get there. GDAL would not install cleanly using basic pip on Windows because it needs pre compiled binaries that pip cannot build from scratch. So, I went through conda forge instead to get it sorted out.

Mosaicking all the tiles together into one single virtual VRT file worked completely fine. It made a lightweight 128KB index file without making copies of any image pixels. But trying to clip tht huge layer to the full EU boundary at the native 10m resolution failed completely. GDAL wanted about 1.75TB of empty hard drive space for tht operation. Huh? And my script did not even catch its own crash properly. It printed out a success message on the screen with absolutely no output file because I had completely forgotten to check the actual return value of the `gdal.Warp()` command.

I dropped tht silly approach entirely because calculating country level land percentages does not actually require pixel perfect continental rasters anyway. Next, I tried calculating zonal statistics straight off the virtual VRT file at native resolution, but my code smashed into a corrupted tile. One downloaded file had genuinely incomplete pixel data inside because my WorldCover script does not do byte level size checking the way my DEM tool does. My computer then threw an out of memory error even after I deleted and re downloaded tht bad tile. So I tried resampling the whole map down to 100m, but the script still ran completely out of RAM on at least one large country shape.

Here is wht actually worked on my computer. Resampled the whole dataset down to 500m using a strict nearest neighbor rule. So I chose to deliberately skip any standard averaging math because WorldCover values are jst categorical class codes (nd trying to interpolate a number between "cropland" nd "built up" zones creates a completely meaningless fraction instead of an actual land class). Then, I ran `rasterstats.zonal_stats()` straight against my NUTS boundary layers..... That simple tweak gave me clean, per country land cover percentages across all 11 classes, and my script saved everything out to a single `landcover_stats_by_country.json` file. I learned the exact same lesson here that my earlier hard drive storage failure taught me: you must always match your processing map resolution to wht your final data analysis actually needs instead of blindly sticking to the source data's native resolution.

Entry 11

I processed the six raw ERA5 yearly files n immediately hit a weird format surprise. `xarray` could not open them at all. So I ran a quick check with `zipfile.is_zipfile()` nd found out they were actually regular ZIP archives wearing a fake `.nc` file extension (which is just a silly known quirk of the newer CDS download system). To make things worse, each ZIP unpacked into two completely separate files. One held temperature data nd the other held rain data, split by an internal GRIB field type rule tht I had not planned for. Chose to write a small script called `unzip_era5.py` to handle both of these layers automatically across all six years instead of hardcoding a single year by hand.

So I tried merging the two variables per year nd immediately hit a messy metadata conflict error. The files had an internal bookkeeping coordinate field called `expver` with different numbers inside each file. I dropped tht specific coordinate completely before merging the data. At the same time, I changed the math units to make sense to a regular policy reader (switching Kelvin to Celsius nd meters to millimeters because nobody likes reading raw climate model numbers anyway). Next, I turned the big gridded cells into per country monthly stats using `rasterio`'s geometry masking tools straight against my NUTS boundary maps. My code skipped a few tiny island territories because their land shape is too small to touch even a single grid cell at ERA5's rough 31km resolution (which is a normal data limitation, not a code bug). So I ended up with exactly 5,472 country month records across the whole six year database.

Entry 12

The GDP data was the easiest thing to process in this whole batch. I did not need any geospatial mapping tools, no pixel resampling, nd there was zero memory pressure on my RAM (jst plain text decoding of Eurostat's custom JSON-stat format). So I chose to read up on the official format specification sheet rather than guessing the structure by hand. The format stores dimensions as ordered category indices, and the actual values sit inside a flat dictionary keyed by an encoded integer. To turn this back into normal data rows, you have to decode it back into per dimension indices using continuous modulo nd floor division math. I wrote tht raw decoding math logic by hand instead of installing some external third party JSON-stat library (mostly because I want my code to stay fully transparent n auditable rather than letting it be a complete black box). So I flattened the final output table into exactly 18,470 records. This includes every single NUTS level together rather than jst country level lines because the source dataset mixes everything up in one file. Finally, I wrote the data straight to a CSV file using Python's built in `csv` module since I did not even need a big library like pandas for something this simple.

Entry 13

The NDVI script took me two real attempts before it finally worked. My first try used CGLS's direct machine to machine API (which was the exact route I picked earlier jst to stay away from Sentinel Hub's complex OAuth setup). The login part worked perfectly fine via a four step JWT bearer token system. So I verified it was working because my web requests started throwing data errors instead of login errors. But then every single file download failed completely. I fixed my bounding box format, tried splitting the data query into a 15×15 spatial grid to bypass server size limits, nd even tried passing NUTS country shapes instead of bounding boxes. Absolutely nothing worked because the real issue was a hidden system architecture problem: this specific dataset does not even sit on the CLMS servers at all. It is jst a pointer link referencing a Sentinel Hub "Bring Your Own Collection" database entry (so no amount of perfect CLMS request text was ever going to download anything).....

So I changed plans nd rebuilt the tool straight on Sentinel Hub using the `byoc_collection` ID number I found while tracking down the CLMS failure. So I set up a brand new OAuth Client, wired up the login checks through the standard Client Credentials flow, n switched to using their Statistical API instead of downloading huge raw images. This was a smart choice because tht API calculates zonal statistics right on their servers n sends them straight to me as a clean JSON string (which completely combines the downloading nd processing tasks into one single step). I hit a small evalscript code bug early on because my `dataMask` variable needed a matching output configuration line. Then I noticed tht the first set of successful numbers coming back sat in the 0 to 250 range instead of looking like real physical NDVI values which must run from -1 to +1. This meant I was getting raw digital numbers rather than scaled NDVI. So I chose to read the official CGLS product manual instead of guessing a conversion formula by hand. I verified the scaling math is exactly `real NDVI = DN × 0.004 - 0.08`, nd then I put that equation straight into the server side evalscript itself to get correct values automatically.....

Ran the full batch script across 27 countries for all 6 years. Exactly 26 countries came back completely clean on my very first try. But France failed for every single year with a bad resolution limit error code. Traced this crash down to its official NUTS map shapes (which unfortunately include far away overseas islands like French Guiana, Réunion, nd Martinique that stretch out the full bounding box area until the server gives up). Huh, wasnt expecting tht. I chose to clip France's map boundaries to the exact same European bounding box limits tht every other country already uses. Then, I re ran the script for France all by itself, nd it passed with no errors. Yay. My full batch data folder now holds exactly 162 clean records covering all 27 countries across the entire six year window.

Entry 14

So I used a quiet stretch of time (while waiting on my big data downloads for NO2, DEM, nd the population files) to actually check if the four datasets I already finished—weather metrics, land cover grids, regional GDP, and NDVI stats—could even merge together into one clean table. So I did not want to jst assume everything would work perfectly and then hit a wall during my final analysis step. It is a really good thing I checked. My weather data nd land cover files both included non EU countries because those specific scripts had accidentally run through the big 39 country NUTS file instead of using my filtered EU-27 roster. Even worse, my GDP table was mixing multiple administrative NUTS levels together alongside different measurement units inside the exact same unfiltered data column.

Built a shared `filter_eu27.py` utility tool so tht every single dataset needing EU-27 filtering can pull from one centralized place instead of re deriving the country list over nd over again. Then, I filtered my weather, land cover, n GDP tables down to size, but I chose to keep a `_full` file backup of each original dataset just in case a broader regional scope turns out to be useful for me later on. My first pass at the GDP filter produced a weird 1,134 records instead of the expected 162 rows. This mistake led me to find out that the exact same country year row had seven completely different GDP values ranging from a tiny 123 to over 3.5 million (because there were multiple measurement units like per capita euro, million euro, million national currency, n several PPS variants tht my original decoding logic script had silently dropped from the final output table). Wow, didnt see tht coming.

So I fixed the decoder code to keep the unit label string, picked `MIO_EUR` (which stands for absolute GDP at current prices) as my project baseline standard, nd re filtered the data rows correctly to exactly 162 records. Along the way, I made a really stupid mistake of restoring my GDP table from an old file backup tht predated the unit fix which jst re introduced the exact same bug back into my data folder (the actual fix was re running

the full decoder script from the source files, not restoring old backups). My final scope check across all four cleaned datasets finally came back clean and totally consistent.

Entry 15

I applied the exact same Sentinel Hub Statistical API pattern to my NO2 data that had just worked for my NDVI script. So I chose this route because my original RPRO/HARP raw acquisition pipeline (which I had fully built and verified at a small scale) turned out to be totally impractical at the real 27 country × 6 year scale. Running tht setup would mean handling roughly 55 to 60 orbital files a month across 72 months (nd jst doing the file skip checking checks alone was eating up several minutes per run). Rather than forcing tht heavy code through, I moved to server side aggregation the exact same way I did for NDVI.

Hit one evalscript error early on. My data showed tht NO2's quality filtering is not a per pixel band layer the way I had originally assumed. It is actually a request level `processing.minQa` parameter, nd Sentinel Hub's own documented default for tht setting value is 75 (which happens to line up exactly with the ESA recommended threshold I had already locked down for this project). No compromise was needed there at all. Reused the exact same bounding box geometry clipping code that I had built for France's NDVI fix earlier. The full 162 request batch (covering 27 countries × 6 years) came back with absolutely zero failures on my very first attempt. There was no France style code anomaly this time because the spatial fix was already baked straight into the loop before I ran it. The original raw RPRO pipeline script stays in the repo completely untouched nd functional at a small scale jst in case a future task calls for raw Level-2 file access again.....

Entry 16

So I ran the EU-27 consistency filter tool against my GDP and NO2 tables. So I cleared out two small bugs along the way. My `filter_gdp()` function was still trying to filter data on a `unit` column tht did not even exist anymore (since an earlier coding session had already fixed tht unit mixing problem straight at the source). I removed tht obsolete filter condition and the drop line from my script, nd the GDP table filtered out completely clean at a perfect 162 to 162 rows. My NO2 data also came back exactly at 162 to 162 records..... This confirmed tht my data acquisition step had already been EU-27 scoped right from the start (so the filter step here was jst a formal confirmation check rather than an actual data correction).

So I also made a big, real decision about the Population dataset. Only the 2019–2020 files have ever downloaded successfully on my machine. Completing the 2021–2024 years would mean integrating a completely different WorldPop data source that I have not tested yet. Since every other single dataset spans the full six years, I decided that the Population data will not go into my core causal model as a control variable (because a variable with four full years of missing data would do way more harm than good in a regression setup). It stays in the folder as a supporting, descriptive dataset instead. The DEM map filter is still blocked until its massive file download finishes, n the NO2 dataset's raw nested structure still needs a flattening script before it can join the master merge table. Both tasks are carried forward to my next session.

Entry 17

So I flattened the raw NO2 dataset from its nested Sentinel Hub structure into a plain per country year month table. My very first run came out exactly 162 records short of the expected 1,944 rows..... That number was way too clean to be a random coincidence because it is the exact total of my country year pairs. I grouped the

output rows by country year nd found tht all 162 pairs had exactly 11 months of data instead of 12. I ran a second check by sorting month numbers nd confirmed it precisely: December was completely missing everywhere, across every single country year, with zero exceptions. Huh?

Traced this bug down to the acquisition script's time range parameters. Both date blocks were ending at `{year}-12-31T23:59:59Z` (which is exactly one second short of a full month interval under Sentinel Hub's half open `[start, end)` window logic). To get a real December interval, the date string needs to end at `{year+1}-01-01T00:00:00Z`. So I fixed both time range code blocks, re ran the full 162 request data acquisition loop, nd re flattened everything. This time it produced the full 1,944 rows with all twelve months present everywhere in the data folder. The perfect uniformity of the original gap (100% of rows missing exactly one specific month) was my actual diagnostic clue here. A random or partial gap would hve pointed me toward a completely different bug somewhere else.

Entry 18

I closed out the last few processing gaps n assembled the real master dataset. I found three separate bugs while doing it.

The DEM data processed cleanly on my very first attempt. My script turned 860 raw tiles into one single virtual VRT file, and I resampled it down to 500m using bilinear interpolation this time rather than a nearest neighbor rule. Chose this because elevation is a continuous metric (meaning averaging between grid numbers is scientifically valid here, unlike the land cover dataset's categorical codes). Then, I filtered the output row count down from 39 to 27 records without any issue.

My NDVI table turned out to have the exact same December data gap tht I had jst fixed inside the NO2 code. It was the exact same Sentinel Hub half open interval issue which happened to be independently present because my NDVI and NO2 layers are handled by completely separate script files. I applied the identical date string fix, re ran the data acquisition loop, nd it came back completely clean.

The master merge process itself surfaced three separate bugs. First, the land cover values landed inside the merged table as a literal stringified dictionary text instead of individual numeric columns. Fixed this by writing code to detect tht nested structure nd flattening each single land class out into its own proper data column. Second, I hit a nasty `ValueError` crash on the CSV writing step because different countries hve different sets of land cover classes present on the ground. This meant deriving my CSV fieldnames from jst the very first row broke immediately on any later row with an extra land class inside it. So I fixed tht by collecting the complete union of all keys across absolutely every single row before letting the script write the file. Third, nd this is the one that took me the longest time to track down: my `avg_temp_c` variable was null in literally 100% of the rows while `avg_precip_mm` (which I pulled from the exact same lookup dictionary inside the exact same code path) was fully populated. That was a total logical impossibility tht ruled out a simple merge logic bug outright. Huh?

Traced it down to the raw ERA5 file actually containing 3,888 records instead of the expected 1,944 rows. This happened because the temperature nd precipitation source files carried slightly different internal timestamps for the exact same calendar month, n combining them without explicit time alignment forced xarray to treat those rows as genuinely separate time steps (creating two half populated rows per month instead of one complete one). Forced the precipitation dataset's time coordinate onto the temperature dataset's layer before combining them. Then I re ran the full ERA5 pipeline script. My final master dataset came out with exactly

1,944 rows and 21 columns, showing zero missing values anywhere except for the legitimate satellite retrieval gaps inside the raw NO2 nd NDVI tracks themselves.

Entry 19

I built the actual causal model code. It took a massive research design detour plus a genuinely nasty conda environment failure to get a trustworthy result out of it. Ahh, tht's a tough one.

Difference in Differences was my obvious first choice. But the actual setting of my dataset does not fit tht layout cleanly (because the European Green Deal rules apply to all 27 EU countries simultaneously, meaning there is zero natural control group sitting inside my table). Chose to check whether the policy intensity variation numbers across different countries could substitute for a control group. It couldn't. The EU regulations sitting in my policy database apply uniformly across the board with absolutely no country specific tagging. So, I redesigned my entire math setup around a timing based approach instead. I picked the European Climate Law's 30 June 2021 effective date to act as a single treatment cutoff point, nd then I forced country fixed effects n seasonal controls to handle the rest of the identification work.

My first code implementation (using the `linearmodels` library) ran without throwing an error but produced absolutely no output text on my screen..... It was a complete silent failure. I checked the math and found out tht entity fixed effects already fully absorb any time invariant variable (like my constant elevation stats or land cover percentages), so including them explicitly alongside entity effects created a perfect collinearity mess. So I removed them as explicit regressors from the formula. But the silent failure kept happening anyway for a much deeper mathematical reason. Because I had full time fixed effects turned on alongside a treatment variable tht stays identical across all 27 countries for any given month, the `treatment` column became mathematically indistinguishable from the time fixed effects themselves. So I swapped out the full time effects for coarser calendar month dummies instead. This step let the treatment variable's before/after variation actually be estimable (though it comes at the documented cost of no longer controlling for one off time shocks like the COVID era lockdown disruption).

Even after both fixes, the model kept dying silently with no Python exception text at all. I chased this bug super hard. The system exit code pointed straight to a Windows level crash sitting below the interpreter itself (not a normal Python error). So I swapped out `linearmodels` for `statsmodels`, but I got the exact same crash. Stripped out the clustered standard errors from the script, but it still crashed. So I chose to force a single threaded execution routine just in case it was an Intel MKL threading problem, but the terminal threw the exact same crash.

Rewrote the whole script to bypass the formula API entirely so I could build the design matrix by hand, but it still crashed. Reduced the whole code block all the way down to a basic `numpy.linalg.lstsq()` check on a random matrix of the same shape, but it still crashed on my screen. Next, I tried an alternate math solver tool, nd then I isolated the problem down to a bare matrix multiplication step with zero solving steps included. It still crashed. That finally nailed the issue down for good. It was a broken Intel MKL backend setup inside my conda environment (not a single line of my actual project code was wrong). Reinstalling both numpy and scipy with the special `nomkl` package (which forces the system to use OpenBLAS instead of MKL) fixed the crash immediately.

With the environment actually working, the model ran completely clean. My data showed a treatment coefficient of -2.285×10^{-6} with a p value of 0.026. The 95% confidence interval came out entirely negative, the R^2 sat at 0.39, nd my total sample count was exactly n=1,748 rows. I caught a statistically significant drop in

NO2 levels following the passage of the Climate Law. This is the very first real mathematical result coming straight out of this project's core question. Wohoo.

Entry 20

Okayyy so here's the thing — I pushed the model way harder before trusting any numbers. Ran the exact same code setup against NDVI to check a second outcome..... It gave me a small negative coefficient with a p value of 0.128 which is completely not significant. This read as highly plausible rather than disappointing (because the Climate Law's direct rules like emissions trading n industrial laws connect to NO2 gas levels way faster than they touch overall vegetation health, which usually responds to deep land use policy shifts on a much longer timescale).

Then, I ran a fake placebo check straight on the NO2 result. Used the same model code but artificially shifted the treatment date backward to 30 June 2020, which is an arbitrary date with absolutely no big policy changes. The script came back showing it was *more* significant than my real result (p=0.002 versus 0.026). Huh? This is a clean, total placebo check failure. A model tht finds a fake "significant change" at a random date cannot be trusted on the real date either. So I chose to add a straight linear time trend column into the regression formula as a diagnostic test. With tht included, my real date coefficient value dropped all the way to p=0.408, meaning it was no longer significant at all. That confirmed exactly what the placebo crash was hinting at: my original regression setup was just picking up a general multi year downward trend in NO2 across the whole time window, not a sudden shift caused by this one law.

This is not a coding bug or a data loading problem. It is a structural identification limitation in my research design. With absolutely no untreated comparison group inside my project data folder, a single cohort and time only treatment design mathematically cannot separate "the law caused this" from "NO2 gas was already falling for unrelated reasons n just kept falling through the treatment date anyway." The only real fix here is to add a true control group to the model instead of just writing a better regression spec. Picked the UK, Norway, nd Switzerland for this (they are geographically n economically comparable non EU European countries tht are completely not subject to the Green Deal rules). Sat down nd scoped out exactly what this new step would require: downloading completely new boundary map files (using GADM data layers because the NUTS system is strictly EU specific), extending my satellite data acquisition loops for NO2, NDVI, nd weather variables across three more countries, and hunting down a second separate GDP data provider since Eurostat does not cover non EU territories in its regular tables.

Entry 21

I built the actual control group. The boundary shapes for the UK, Norway, nd Switzerland came from the GADM database at the country outline level, and I verified them for correct codes, valid geometry, nd matching CRS before I trusted them. I chose to build `country_boundaries.py` to act as a single shared code interface tht loads spatial geometry from either NUTS (for the EU-27) or GADM (for the control group countries) through the exact same function. This standardized both sources to a consistent two letter country code convention nd reused the bounding box clipping fix that I already proved on France.

Extending the NDVI script to run across all 30 countries surfaced two new failures. France crashed again because the bounding box clipping was not enabled by default in this script, and I hit a brand new failure for the UK. It was a `COMMON_BAD_PAYLOAD` error code that I traced down to the sheer geometric complexity of its unclipped GADM map shape (which has about 2,562 separate polygon parts across all its small coastal

islands). Huh? Enabling the bbox clipping fixed France immediately. The UK needed a second fix because clipping its already complex map shape against a simple bounding box produced a mixed `GeometryCollection` layer (which mixes up degenerate points n lines with real polygon area, something the Sentinel Hub API rejects outright). So I extended the clipping logic script to filter down n keep only the polygon components, nd the UK data finally went through clean. The NO2 script extension to all 30 countries hit zero failures on my very first attempt because both fixes were already sitting in place before I ran the code.

My weather data needed a different kind of code fix. Norway nd Switzerland were already present inside my original NUTS file because they are EFTA members. Pulling them again from the GADM file caused a double counting issue (which produced exactly double the expected record rows for those two specific countries). So I fixed this by forcing the script to explicitly skip any NUTS map entry tht matches a control group country code. Next, I pulled the GDP numbers for my control group from the official World Bank API instead of Eurostat (since Eurostat does not cover non EU countries in its files). Added an explicit, documented EUR/USD currency conversion step using annual average rates. This is an acknowledged approximation step, but it is totally acceptable for a baseline control variable rather than a primary tracking metric.

With all four data tables extended n a fresh `treatment_group` column added to the master file, the real two group DiD model finally ran completely clean..... My code found an interaction coefficient of -1.40×10^{-6} with a p value of 0.632, n the confidence interval spans straight across zero. Once you actually measure the EU against a real non EU comparison group, there is absolutely no statistically distinguishable EU specific NO2 reduction happening on the maps. Whatever decline happened inside the EU-27 borders looks like it is jst part of a much broader trend shared with comparable non EU neighbor countries (not something special caused by this one piece of legislation). This was not the happy result I might have hoped to find when I started this project, but it is the exact one tht my data validation process actually supports.

Entry 22

I ran one more robustness layer on my control group result. Set up an event study script, splitting up the single before and after comparison into exactly 23 separate quarterly coefficients (covering 2019Q1 to 2024Q4, measured relative to the 2021Q2 baseline) rather than looking at one single averaged effect. This answers two big questions tht a single average number cannot tell me: did the EU-27 and my control group already look different before the policy treatment (which would completely break my model's core assumption anyway), and could a real but delayed effect have gotten averaged away across the whole multi year post period?

All 23 quarters came back completely non significant at this stage (this was before the Balkan countries were added to the control group and before the NO2 data were corrected in Entry 32; the later results are in Entries 31 to 36). My data showed p values scattered randomly between 0.18 nd 0.94 with absolutely no discernible pattern before or after the treatment quarter. That is two pieces of good news at once for my code. There is zero pre treatment divergence (supporting the parallel trends assumption tht this entire DiD design leans on) nd no hidden delayed effect either (so my null result is not just an artifact of averaging things out). Combined with my earlier placebo test and the control group correction, tht is a solid three part validation sequence backing up the same conclusion from three totally different angles which is a meaningfully stronger basis than any one of them alone. Module 8's core causal inference work is totally done at this point: there is no statistically distinguishable EU specific NO2 effect once you genuinely check it against a real comparison group n stress test it three separate ways.

Entry 23

I rebuilt my Python environment from scratch before visualizing my event study coefficients, since matplotlib was not installed yet n I wanted the full dependency list (including `nomkl`, given the MKL crash from a few sessions back) specified cleanly up front instead of installing packages one at a time.

The actual plot came together cleanly. It shows 23 quarterly coefficients as points with confidence interval error bars, a zero reference line, and a vertical line marking the treatment date. Verified it by opening the file directly n walking through it against a checklist since I could not view the image inline in this session. All 23 points are present and correctly ordered on the screen..... Every single interval spans across zero, matching my regression output exactly.....

Entry 24

So I built out the full set of geospatial visualizations on my system. I chose to use a Python only pipeline (geopandas + matplotlib) instead of opening QGIS because every input file was already in a directly usable format. I wanted my final maps to stay completely reproducible and version controlled right alongside the rest of my project code. So I verified each single map by opening the file locally on my machine nd checking it line by line against a specific checklist for wht it was supposed to show (since I could not view images inline mid session).

The NO2 choropleth map needed a full colormap swap partway through. My initial yellow to red scale made several low NO2 countries visually indistinguishable from the blank white map background layer. I fixed this issue by switching the code to use `plasma` (whose low end stays perfectly visible on screen). My land cover map crashed outright with a nasty `nan` RGBA error code. I traced this crash down to the loading function assuming a much flatter JSON structure than my actual nested data layout. Fixed the extraction path inside the code nd added a solid gray fallback style for any future unmatched land class. My GDP map had a real communication problem rather than a standard code bug. GDP data is heavily right skewed across Europe with a handful of huge economies completely dwarfing everyone else on the chart. A standard linear color scale compressed nearly every single country into one single pale color band. I fixed this by applying a log transform step nd switching the map scale to `viridis`. This spread the color scale smoothly across the actual data distribution instead of just highlighting the top few outlier countries (nd I explicitly labeled the colorbar as a log scale so nobody misreads it as raw GDP values).

Ended up with exactly seven visualizations total inside my output folder..... These include the NO2 choropleth map, a 2019 versus 2024 before and after comparison view, a categorical study design map explaining my treatment nd control split setup, a dominant land cover class map, the NDVI choropleth chart, the log scale GDP choropleth layout, and the quarterly event study coefficient plot from my prior coding session..... For the before and after panels, I chose to deliberately force one single shared color scale across both views (which ensures any visible shift on the screen represents a real data change rather than a deceptive scaling artifact). Every single map looks stylistically consistent across the whole set now. They are completely ready to go into my final frontend dashboard.

Entry 25

So I built a Streamlit dashboard to present all of these results interactively across eight pages — home, environmental data, study design, before/after panels, economic context, causal results, methodology nd

limitations, nd about/data with a downloadable master dataset link, including a Plotly based country selectable time series explorer and the regression tables — and pushed the complete project onto GitHub for the very first time as a public, open repository.

Entry 26

The dashboard went live on Streamlit Community Cloud, and I filled in the remaining gaps — visualizing the DEM nd ERA5 control variables that had never made it onto a page, and adding a before and after NDVI panel to match the existing NO2 one.

I tested whether the "globally transferable" claim in my project's design papers was actually true rather than just an empty assertion. Built a standalone test script tht reuses only my existing Sentinel Hub login authentication logic nd evalscript code blocks. I pointed this new script at a simple bounding box over India instead of picking an EU country, making absolutely zero changes to the validated EU-27 pipeline itself. All six years of data came back completely clean with physically realistic NO2 numbers. This test confirms tht my data acquisition architecture genuinely is portable, not just claimed to be on paper.

Entry 27

So yesss, I went through the full code repository end to end ahead of finishing this project properly. I chose to apply the exact same strict statistical checking tht my NO2 results had already gone through to absolutely everything else in the folders instead of just treating that one validation step as covering my whole project work.

Also went through the repository fixing a handful of small housekeeping issues — broken file paths, an incomplete `requirements.txt`, duplicate output files, nd some inconsistent dataset counts across the README nd dashboard — none of which changed any actual result.

The bigger fix I did was purely statistical. Every single causal inference script in my project was using plain OLS standard errors (which understate uncertainty for panel data with repeated monthly observations per country, a well documented math issue). Chose to cluster my standard errors by country across all five regression models. My headline DiD result moved from p=0.632 to p=0.663. The null result got more solid instead of breaking down (so this change was completely safe to apply to my data files). Honestly, tht's wht I expected. My quarterly event study picked up three nominally significant quarters under this clustering setup tht had never shown up under classical SEs. This is close to the roughly one false positive you would expect by pure random chance at this specific sample size with no consistent direction on the charts, and I reported tht plainly inside my log instead of leaving it out to make the results look better.

Next, I ran five more robustness checks straight against this corrected model. I removed the GDP column entirely (nd my final result moved even closer to zero, proving tht GDP was not driving any fake results). Tried a log transformed outcome check (which confirmed tht my null result is not a structural functional form artifact). Also checked treatment date sensitivity, ran a baseline pollution heterogeneity split test, nd added a formal minimum detectable effect math calculation..... This final check quantified wht I had only been describing qualitatively on paper before. My data proved that at 80% statistical power, this specific design can detect an effect of roughly 28% of baseline NO2, while my actual regression coefficient sits at about 4.4% of that baseline (this was recalculated after the 9-country control group and again after the data correction in Entry 32: on the corrected data the MDE is about 5.5% of baseline and the baseline-model coefficient about

4.9%; the current paper does not report an MDE). This is a real, previously implicit model limitation tht I am now stating as an actual hard number.

While extending the clustering setup to every model, I found that my NDVI analysis had never actually gone through the control group correction that my NO2 analysis had. It was still running the original single cohort design tht my placebo test had already proven unreliable for exactly this kind of data problem. I chose to rebuild it completely to match the corrected NO2 design, and it produced a genuinely different result. Where the uncorrected model found absolutely nothing (p=0.128), my corrected two group model finds a real, statistically significant relative NDVI decline inside the EU-27 borders versus the control group countries (showing a coefficient of −0.0210 with a p value of 0.012).

Reported this inside my project as a secondary finding, not as absolute proof tht the Climate Law harmed vegetation outright (since land use changes, raw drought cycles, nd general agricultural policy goals are not controlled for here). Instead, I added it as a real reminder tht validation applied only to a project's headline result can leave something sitting completely unnoticed inside a secondary one.

Entry 28

Reviewed the finished project once more specifically against the kind of questions a reviewer would actually raise. Reviewers always ask about things like control group size, spillover risk into the control countries, how the NDVI finding should be interpreted, or using country level versus finer spatial resolution on the maps. Rather than trying to patch everything at once, I chose to triage each candidate improvement by comparing real coding cost against real value (since I still have six more portfolio projects that need this exact same treatment).....

Here is what I actually did on my machine. I added a new paragraph making the UK's 2020 EU exit an explicit, stated part of my control group justification rather than just leaving it implicit. Then, I added a Future Work section naming the Synthetic Control Method, finer spatial resolution grid layers, formal spatial autocorrelation diagnostics, nd a delayed effect test as specific next steps. This ensures that any obvious "why didn't you try method X" question gets a direct answer from me instead of silence. Also added a clear architecture diagram and a reproducibility section straight to the README file, n I set up a `CITATION.cff` file for eventual publication. Here is what I deliberately did not do: I did not actually build the synthetic control model or run the spatial diagnostics themselves. Each of those is a real, heavy data and modeling effort in its own right (not a simple documentation fix), so I named them as future work instead of quietly skipping over them. No new empirical claims came out of this final pass, just presentation nd documentation improvements sitting on top of my already validated analysis.

Entry 29

So I went back through every single quantitative claim sitting inside my paper. Chose to independently recompute absolutely every figure straight from the master datasets directly rather than trusting numbers that had just accumulated across a lot of separate coding sessions over time. Everything checked out perfectly to the reported precision except for one real inconsistency on my sheet. My two earliest, already superseded single cohort models (the original NO2 result nd the original NDVI result) had been reported with classical, non clustered standard errors. This happened even though every later model in the project uses cluster robust SEs by country. That standard just never got applied retroactively to those first two runs once it was adopted.

I recomputed both of them properly on my machine. The old NO2 number moved from p=0.026 to p=0.041 (which is still significant so it does not change that specific section's conclusion at all). The old NDVI number moved from p=0.128—which I thought meant "no effect"—to p=0.0017, which is genuinely significant. This change completely alters the accurate story here: my original single cohort NDVI model was already picking up a real effect under this project's own stated methodology, not just after the control group correction came along. Of course, my corrected two group model (p=0.012) is still the trustworthy, final reported result for my project. Its real role is better identification by isolating an EU specific effect from a shared regional trend, rather than first time detection.

Everything else matched up perfectly to the reported numbers with no extra changes needed on my drive. My data checks proved that the placebo tests, the linear time trend column, the corrected two group models (both for NO2 n NDVI maps), the quarterly event study chart, all five robustness runs, and the minimum detectable effect calculations matched down to the exact decimal precision. I also spot checked three of the primary paper citations against fresh online searches nd verified tht they are completely real n correctly attributed to their authors. I flagged the remaining ten citations as not individually re verified during this specific round instead of silently faking tht I checked every single line. So I fixed the two inconsistent passages across my paper text, the main project overview, nd the frontend dashboard layout..... Finally, I reframed the NDVI narrative to tell the true story of what happened on my machine: absolutely no underlying source data or regression model logic changed at all, because this fix was purely a standard error consistency cleanup.

Entry 30

I built two more checks against my corrected NO2 model. Both tasks were already named in my Future Work list rather than just leaving them as vague intentions on paper.

So I tackled the synthetic control first. With all three control countries sitting inside my donor pool, Norway's raw NO2 series turned out to be missing exactly 29 out of 72 months (which is a real data acquisition gap on the server, not something random). This problem collapsed my usable pre treatment window down to a tiny 12 months if I chose to keep it in the loop. I dropped Norway from the donor pool entirely rather than trying to impute fake missing values over a 40%-missing series. Keeping jst the UK plus Switzerland leaves 28 out of 30 pre treatment months completely intact in the folder. I fit my convex NNLS weights (giving the UK exactly 21% nd Switzerland 79%) nd added a ridge augmented bias correction step straight on the residual values. My post treatment gap came out with the exact same mathematical sign nd roughly the same size as my main DiD coefficient. Since I only hve two donor countries left, the in space placebo test does not act as a real permutation check (and I said so directly inside my notes rather than dressing it up to look more rigorous than it actually is).

Then I checked for spatial autocorrelation. Built country geometries using my existing boundary map layers and chose to run KNN-4 weights on projected centroids instead of standard contiguity sharing (because several countries in my list are islands tht would end up completely disconnected under a simple border adjacency rule). Running a Global Moran's I test on my raw NO2 pollution levels came back strongly clustered (0.522 with a p value of 0.001), which is exactly what makes sense physically since toxic air pollution does not respect political borders on a map anyway.

The genuinely useful check was testing my main DiD model's error residuals, and tht code came back completely non significant (I=0.020 with a p value of 0.247). This means that my country n calendar month fixed effects are already absorbing the spatial dependence on their own, proving tht my clustered standard

errors are not missing anything tht a heavy spatial regression model would need to fix. I added a Local Moran's I step on top just to see exactly where the spatial clustering sits on the map. My data found a clear high pollution cluster hanging across Benelux, Germany, nd Denmark alongside a distinct low pollution cluster sitting across the Nordic n Baltic regions.

I wrote both of these checks straight into the paper as brand new methodology nd results blocks. Removed the two corresponding data items from my Future Work list since they are now actually done n working on my machine. Finally, I updated my limitations text block to state tht my synthetic control's donor pool is thinner still than the DiD's control group country count, nd I noted tht Norway's messy data coverage gap is a real, acknowledged data quality issue in my folder.

Entry 31

So I expanded my control group from 3 countries to 9. I added Iceland, Albania, Bosnia nd Herzegovina, Montenegro, North Macedonia, and Serbia to the list, plus I did a full clean re fetch of Norway's NO2 data series jst to check whether its annoying data gap was ever fixable. All six new countries already had their NUTS geometry sitting in my existing boundary layer file, so I did not need to do any new map boundary coding. My climate data came completely free from the ERA5 weather grids I already downloaded earlier (I just had to run a wider zonal stats loop over the exact same data files).

Norway's data gap did not fix at all. It is still missing exactly 29 out of 72 months even after a completely fresh, clean re fetch from the server. This finally settles it as a real high latitude satellite retrieval coverage limit, not the stale file download bug I had assumed earlier. Iceland hits the exact same issue at a slightly smaller scale. Chose to keep both of them excluded from my synthetic control donor pool, and I wrote this down plainly in my notes rather than smoothing things over to make the logs look perfect.

Next, I re ran absolutely everything downstream tht depends on the control group instead of jst stopping after the new data collection task. I updated my NDVI script, the quarterly event study, all five robustness checks (which I consolidated into their own separate master script this time since they never had one before), the synthetic control weights, my spatial autocorrelation tests, n every single map or chart plot touching the control group countries..... I chose to do this complete overhaul so nothing was left showing old three country numbers next to my new nine country results.

The extra statistical power actually changed my whole story instead of just tightening the same old null result. My pooled DiD coefficient moved straight to -2.22×10^{-6} with a p value of 0.101 (which sits way closer to conventional significance cuts but still does not make it on its own). Ran a heterogeneity split based on baseline pollution levels, nd it came back highly significant for higher baseline countries specifically (mostly covering Western nd Central Europe with a p value of 0.003 and a coefficient of -5.46×10^{-6}). This is something my old three country setup simply did not hve enough power to detect at all (it had shown a bad p value of 0.245 on the exact same split test). My quarterly event study now shows four significant post treatment quarters on the chart. They are all negative numbers, and they all land neatly inside Q2 or Q3 (which signals a real, physical seasonal pattern rather than scattered random noise the way my earlier three quarter result had been).

My treatment date sensitivity check threw a genuine oddity on my screen. Shifting the policy date six months earlier is itself statistically significant (p=0.021) while my true legal date is not (p=0.101). So I chose to flag tht inside the text rather than picking whichever date looks better for my paper (since it might mean the policy effect's real onset does not line up exactly with the legal date, or it might mean something else entirely). My

log transform check went the opposite direction from every other check here, shrinking down toward zero. This actually makes perfect sense given the pollution drop looks concentrated in a small subset of heavy countries rather than showing up as a uniform percentage decline everywhere on the map (n I noted tht in my logs rather than treating it as a contradiction).

None of this mess completely overturns the pooled null as my headline number because the main interaction term's confidence interval still crosses right over zero. But three independent tests now point in the exact same direction (the baseline heterogeneity split, the quarterly event study, nd the date sensitivity check) which my smaller control group simply did not have the statistical power to distinguish from background noise. I rewrote my paper's headline numbers, the main abstract, and every single affected section to describe the new nine country design directly rather than presenting it as a quick update from three. So I regenerated all 11 affected maps and plot charts, and I wrote this up as an open, partially resolved finding rather than forcing my data into either a flat "still null" or a fake "actually significant" box. So yeah, that's where the project actually stands.

Entry 32

This entry is a correction, nd a big one. I did a full line by line audit of the whole project (every script, every result file, every plot, every sentence in the paper n the reference list) and it turned up a problem in my main outcome data. My NO2 "monthly means" were never monthly means. Sentinel Hub clips every Sentinel-5P request to the last 24 hours of the requested time range, so when I asked for a full month I was actually getting one snapshot from the last day of tht month. Every NO2 number I had reported so far, including the pooled null in Entry 31, was built on those end of month snapshots.

So I had to re download the whole NO2 series properly. My first plan was to keep the Statistical API and just request it day by day, but every single Sentinel-5P request came back with a 500 "Processing error", even the exact request tht had worked for me back in July. I wrote a small diagnostic script (diagnose_sentinelhub.py) tht changes one thing at a time across 14 test requests. It showed my account was fine (Sentinel-2 statistics worked) and tht Sentinel-5P itself was fine through the Process API. Only the Statistical API plus Sentinel-5P combination was broken. So I rebuilt the acquisition on the Process API instead. Now it pulls one raster per day over the whole study box at 0.1 degree, keeps only QA 0.75 or better pixels, masks each of the 36 countries locally, and pools all valid pixel days into a true monthly mean. Tht is 2,192 daily requests for 2019 to 2024. The run saves a daily log as it goes, so when 10 days failed with a server 503 I just ran the same command again and it only fetched those 10. I also re fetched NDVI for all 36 countries in one clean file (with orbit mosaicking, so it is a real monthly mean too).

One thing from Entry 31 I have to take back. I wrote tht Norway's 29 missing months were a real high latitude retrieval limit. They were mostly not. With real daily data Norway is missing only 4 months (Iceland 9, Finland 6, Estonia 1). The big gap came from the snapshot method, because one single day in a northern winter often has no valid pixels at all. So Norway nd Iceland are back in my synthetic control donor pool, which now has all 9 control countries.

Then I re ran everything downstream, and the results changed. My pooled DiD is now -1.52×10^{-6} with a p value of 0.013, which is about 4.9% of the EU's pre treatment mean. The drop sits almost entirely in the higher baseline countries (-2.85×10^{-6}, $p = 0.002$) while the lower baseline group shows nothing ($p = 0.758$). On its own tht looks like a clean significant policy effect. It is not, and the other checks say so pretty clearly. In the event study 7 of my 9 pre treatment quarters are significant. Every fake treatment date I tried (12 nd 6 months

earlier, 6 nd 12 months later) is also significant. And when I add an EU specific linear trend to the model, the DiD coefficient flips to $+8.67\times10^{-7}$ with p = 0.243 while the trend itself is significant (p = 0.016). The synthetic control gives a much smaller gap (-8.10×10^{-7}) and the EU ranks only 5th out of 10 in the placebo test. So what my data actually shows is tht EU NO2 fell about 5% more than the comparison group over these years, but as a slow EU wide trend tht was already running before the Climate Law, not as a break at the law's date. I cannot credit the law for it, nd the paper now says exactly tht. The NDVI result (-0.0194, p = 0.005) stays in as an exploratory association only.

The audit caught smaller things too. My Moran's I test on the DiD residuals was not really testing anything, because country fixed effects force each country's average residual to zero. I now compute it month by month, and the residuals are spatially clustered in 59 of 72 months, so my earlier line about the fixed effects absorbing the spatial dependence was wrong nd I removed it. The ridge term in my synthetic control was doing nothing, so I replaced it with a plain intercept adjustment. I fixed the wording around the law's dates (adopted 30 June 2021, in force 29 July 2021) but kept the model cutoff where it was, so July 2021 is still the first post treatment month. In the reference list I fixed one wrong citation, one misspelled author name, nd a few references tht were listed but never cited in the text. My old single cohort scripts still run on the original data by default so the historical numbers in the paper stay traceable, with a --corrected option to run them on the new data.

I also redid the India transferability test. The old version used a plain rectangle tht covered neighbouring countries nd a lot of ocean. Now it uses India's actual GADM boundary with the same daily method. All 72 months came back with data, the monthly mean ranges from 2.27×10^{-5} to 4.16×10^{-5} mol/m², and every value sits inside the middle 90% of my EU country month values. It only shows the pipeline runs outside Europe, nothing more.

Finally I rewrote the abstract, results, discussion n limitations in the paper, plus the README, the executive summary, the data sources file nd the dashboard pages, and regenerated every plot, the interactive maps and the maps PDF so nothing still shows the old snapshot numbers. In short, my earlier headline was wrong because the data under it was wrong, nd the corrected headline is less exciting than a significant p value makes it look. A relative decline tht I can measure but cannot pin on the law is what the evidence supports, so tht is what I am reporting.

Entry 33

After the correction in Entry 32 I asked myself a harder question. If an editor picked up this paper, what would make them reject it on the research alone? I listed the weak points. Some of them are about the question itself (the Climate Law has no direct mechanism on NO2, nd the contribution is thin after the correction) and those need a bigger rethink. But four of them were technical and fixable right now, so I did those first. They all live in one new script, causal_inference_identification_checks.py.

First the model itself. My headline model used one seasonal cycle for all 36 countries, and the event study had already hinted tht the EU and the comparison group do not share the same seasonal cycle. So I switched to year month fixed effects and gave the EU its own calendar month effects. The estimate barely moved (-1.34×10^{-6}, p = 0.016). What did change is the pre trend problem. In Entry 32 I was worried about 7 of 9 pre treatment quarters being significant. With the EU's own seasonal cycle in the model, the pre treatment blocks are not significant anymore (joint p = 0.292). So most of tht was seasonality, not a real pre trend. I also downloaded ERA5 wind speed and boundary layer height, because NO2 columns depend a lot on how well

the air is ventilated. Boundary layer height matters a lot for NO2 on its own ($p < 0.001$) but adding both did not move my estimate (-1.36×10^{-6}, $p = 0.016$).

Then inference. I only have 9 comparison countries, and cluster robust p values can be too optimistic with so few. So I ran a wild cluster bootstrap (9,999 replications) and randomization inference (2,000 random reassignments of the EU label). The bootstrap agrees with my original p value (0.018). Randomization inference does not ($p = 0.16$). I am reporting both, because picking the one tht looks better would be exactly the kind of thing this project is supposed to avoid. Dropping any single comparison country does not change the result, and it holds separately against Norway, Iceland, Switzerland nd the UK, and against the Western Balkans.

Then COVID and the energy crisis, which both sit inside my study window. I added the Oxford stringency index as a control (it covers 34 of my 36 countries, not Montenegro or North Macedonia) and it changed nothing. Dropping the whole COVID window makes the estimate bigger, not smaller (-2.25×10^{-6}), and so does dropping the energy crisis months (-2.08×10^{-6}). So neither of them is producing my result.

I also had one worry about the satellite data. The TROPOMI NO2 processor had a big version change on 1 July 2021, which is literally my treatment date. If my series mixed old nd new versions, I would have an artificial jump exactly where I am looking for an effect. I checked 24 sample days across the six years. Everything up to July 2022 comes from the reprocessed v2.4.0 record, and after tht from operational v2.4 to v2.7. So there is no version break at my treatment date. There are small updates after 2023 which I cannot fully rule out, and I wrote tht into the limitations.

Now the part tht does not go my way. When I split the post period into 12 month blocks, the first two years after the law show nothing at all ($p = 0.98$ and $p = 0.44$). The whole effect comes from July 2023 onward (-2.18×10^{-6}, $p = 0.008$). If I drop 2023 and 2024 the estimate is basically zero. Significance also depends on the GDP control (without it $p = 0.074$). So the result is sturdier than I expected against seasonality, weather, COVID nd the choice of comparison countries, but the timing problem is now even clearer than before. Whatever lowered EU NO2 relative to its neighbours did it about two years after the Climate Law.

I added all of this to the paper as a new section (4.10), updated the abstract, discussion, limitations nd conclusion, and carried it through the README, executive summary, data sources nd the dashboard results page. I also added four references for the new methods and checked each DOI. My position now is tht my estimate is real enough to survive most checks, but it does not belong to the Climate Law's date. Tht points to the next step, which is to stop treating the law as the treatment and look at what actually changed from 2023.

Entry 34

At the end of Entry 33 I said the next step was to look at what actually changed from 2023. To do tht I needed to see inside the countries, because a country mean cannot tell me whether the drop is in big cities, around power plants, or everywhere. So I went back to the satellite data one more time. My country series only kept each country's daily total and threw the pixels away. This time I saved every daily raster for the whole study box (2,192 days, 0.1 degree pixels) nd built a monthly grid from them. Before using it I rebuilt the country monthly means from the grid and compared them with my existing country series. All 2,572 country months match exactly, so the grid nd the country data are the same data at two different scales.

Then I asked a simple question. Does NO2 fall more where the EU is, or where the pollution was high to begin with? I sorted every pixel into eight classes by its pre treatment NO2 level and looked at the change from July

2023 in each class, separately for EU pixels and for pixels in my nine comparison countries.

The answer was not what I expected. The decline grows steadily with how polluted the pixel was, and it does this in both groups. The cleanest pixels barely moved. The most polluted ones lost more than 20%. And at the same starting level the comparison pixels fell as much as the EU pixels, sometimes more (for example -18.8% vs -19.0% in the 5 to 7 class, and -23.0% vs -21.4% in the highest class). Across all 51,072 pixels the change is about a quarter of the pre treatment level (R^2 = 0.70), and once I account for tht, being in the EU makes no difference (p = 0.221).

So I re ran my DiD on country by class by month cells. Without matching on the starting level it gives me back my country estimate exactly (-1.34×10^{-6}, p = 0.012). When I only compare EU pixels with comparison pixels of the same class, it drops to -0.21×10^{-6} with p = 0.341. About 84% of my estimate is gone. I also defined the classes from 2019 alone nd estimated on 2020 to 2024 so the two do not overlap, and got the same thing (-0.20×10^{-6}, p = 0.401).

What this means is tht my country level result was mostly a composition effect. EU countries simply contain many more highly polluted places than my comparison group (Norway, Iceland nd the Balkans are mostly clean). Polluted places fell the most everywhere in Europe. A country average turns tht into something tht looks like an EU effect. None of my earlier checks could catch this, because every one of them used country averages. It is not a significance problem, it is a unit of analysis problem.

One limit remains. Almost all the highly polluted comparison pixels are in the UK, with a few in Switzerland nd Serbia. The UK kept EU derived vehicle and industry standards after it left. So this check separates EU membership from pollution level. It does not separate EU made regulation from no regulation.

I added this as Section 4.11 in the paper with two new figures (the change by starting level, nd a pixel map of the change), and updated the abstract, discussion, limitations nd conclusion, plus the README, executive summary, dashboard and the maps PDF. The map itself says it clearly. The decline covers the polluted belt from England through the Low Countries nd western Germany down to the Po Valley, plus the big cities, and it does not stop at the EU border.

This changes what the paper is about. I started by asking whether the Climate Law lowered NO2. What I can actually show now is how a country level satellite DiD produced a significant "policy effect" tht survived a long list of robustness checks, and then disappeared once I compared like with like at pixel level. Tht is a less flattering result for my original question, but I think it is the more useful one.

Entry 35

I went too far in Entry 34, nd this entry is me walking part of it back. After the pixel level result I wrote tht NO2 fell "equally inside and outside the EU". Before sending the paper anywhere I wanted the whole project checked again from zero, without trusting my own summary of it. So I had it reviewed four ways (as an editor would read the paper, a number by number audit against the code, a methods review, nd a reference check). The good part is tht every script ran and every headline number reproduced. The bad part is tht the methods review found my "equally" claim does not hold, and I confirmed it myself.

My matched estimate (-0.21×10^{-6}, p = 0.341) was an average of two opposite things. Against the UK, Norway, Switzerland nd Iceland it is $+0.20\times10^{-6}$. Against the Western Balkans it is -0.38×10^{-6} (and -1.08×10^{-6} from July 2023, p = 0.014). When I compared the two halves of my own comparison group with each other, they differ

by -1.00×10^{-6} ($p = 0.014$), which is more than the EU differs from either of them. So "no difference between EU and non EU" was really a plus and a minus cancelling out.

So I looked at the polluted pixels country by country, nd the picture is much clearer tht way. Netherlands -25%, Belgium -22%, Germany -21%, UK -20%, Switzerland -18%. Then Italy -13%, Poland -11%. Then Hungary, Bulgaria, Romania, Croatia, Greece, Bosnia nd Serbia all between 0 and -6%. It is a gradient from north west to south east, and it does not care about the EU border in either direction. The UK and Switzerland are outside the EU and look like Germany. Romania, Bulgaria, Croatia and Greece are inside the EU and look like Serbia.

I tested tht directly. Inside the north west, EU members vs the UK nd Switzerland: -0.41×10^{-6} ($p = 0.44$). Inside the south east, EU members vs the Western Balkans: $+0.22\times10^{-6}$ ($p = 0.48$). But inside the EU, north west members vs south east members: -1.43×10^{-6} ($p = 0.016$), and -2.79×10^{-6} from July 2023. So the border between regions matters and the EU border does not. My country level "EU effect" came from comparing an EU average dominated by the polluted north west with a comparison group tht is mostly clean or south eastern.

The review also found things about my country level estimate tht I had never tested because I never thought of them as choices. Much of it sits in winter. Without November to February it drops from -1.34×10^{-6} to -0.81×10^{-6} ($p = 0.11$). Winter is exactly when the satellite has the fewest good retrievals. Four of my country means in December 2023 are negative (Denmark, Sweden, Latvia, Lithuania), which is physically impossible, and dropping just tht one month shrinks my "late decline" block a lot. My "nothing for two years, then a drop from July 2023" story from Entry 33 also depends on the winter months. Without them it is a slow drift tht starts before the law. Then weighting. I gave every country the same weight, so Malta with 3 pixels counts as much as Germany. Weighted by area the estimate is -0.96×10^{-6} ($p = 0.11$). And GDP. I used it in nominal levels, and with log GDP the estimate is -0.94×10^{-6} ($p = 0.21$).

Two smaller ones. My randomization test used the raw coefficient, where the usual way is the t statistic, and tht gives $p = 0.046$ instead of 0.17. And my synthetic control fitted the weights first nd shifted the level after. Fitting both together puts the EU 2nd of 10 instead of 5th. So in both of those I had been a bit too harsh on my own result, nd in the others too generous. I am reporting all of it.

Everything is in one new script (causal_inference_stress_tests.py) and I added Section 4.12 to the paper with a new figure, corrected the abstract, discussion, limitations nd conclusion, and fixed the same sentences in the README, executive summary and dashboard. I also fixed a few factual slips the review caught (the UK was a member in 2019, not in transition, nd Bosnia only became a candidate in December 2022).

What I can say now is narrower than last time but I trust it more. NO2 fell a lot in the polluted north west of Europe and very little in the south east, on both sides of the EU border, and a country level comparison turned tht into an EU effect. I cannot say why the two regions differ, nd part of the size of it comes from winter data I do not fully trust. The editor style review was also blunt tht the paper itself still reads like a diary of this project instead of an argument, with pieces of three different conclusions left in it. Tht is the next thing to fix, and it means rewriting it around this finding instead of patching it again.

Entry 36

I rewrote the paper from the first line. The old one told the story of my project in the order it happened, nd an editor reading it could not tell what the argument was. The new one asks one question, what does a country

level DiD on satellite NO2 actually identify, and uses the Climate Law as the case. New title: "What Does a Country-Level Difference-in-Differences on Satellite NO_2 Identify? Baseline Composition, Winter Retrievals and the European Climate Law". NDVI, India nd the project history are out of the paper. They stay in the repo and the dashboard.

Before writing I had the first rewrite reviewed again, and it caught me overclaiming a second time. I had written tht the decline is "regional, not institutional" nd "not an effect of EU membership". My own north west contrast has only two comparison countries, and its interval (-1.55 to +0.73) is wide enough to contain my whole country level estimate. So the right sentence is tht the data do not need an EU border effect, not tht there is none. I removed tht phrase everywhere, in the paper, README, executive summary, citation file and dashboard.

I also changed how I compute p values. With 36 clusters, nd only 9 on the comparison side, the normal distribution is too generous, so every current script now uses a t distribution with clusters minus one degrees of freedom. Nothing flips, but everything gets a bit weaker. The main model goes from p = 0.016 to 0.022, the first model from 0.013 to 0.018, the late block from 0.008 to 0.012. With the stricter test only 5 of 9 pre treatment quarters are significant, not 7.

Three new checks went into causal_inference_stress_tests.py.

First, random assignment. I split my 36 countries into 27 and 9 at random, 999 times, and ran the same DiD each time. The estimate follows the pre treatment gap between the two random groups almost on a line (r = -0.81). Whichever group starts more polluted gets a negative "effect". 9.6% of the random splits are significant at 5%, about double what it should be. My real EU assignment sits right on tht line. This is the clearest picture I have of the composition problem, and it is now Figure 3.

Second, I had been saying the matched estimate (-0.21) is smaller than the unmatched one (-1.34) without testing the difference itself. A cluster bootstrap gives -1.13 with a 95% interval of -2.18 to -0.28, so the drop is real.

Third, Turkey. My rasters already cover western Turkey, Ukraine nd Kosovo, I just never used them. Turkey has about 1,000 polluted pixels (Istanbul, Ankara) and they did not fall at all (-0.3%). So "polluted places fell everywhere" is not true either. Then the uncomfortable part: when I match EU pixels against Turkish pixels of the same starting level, the EU difference does not go away (-1.43×10^{-6}, north west members -2.16, south east members -0.30). It is one comparison country, so I cannot give an honest p value nd I do not report one. But it means matching removes the EU difference against the UK, Switzerland and the Balkans, and does not remove it against Turkey. What the matched model gives back depends on where the comparison pixels are. I put tht in the paper as it is, because hiding it would be the same mistake I made in Entry 34.

I also refitted the synthetic control with weights nd level shift together (gap -1.45×10^{-6}, 2nd of 10), cleaned the figures (country borders on the pixel map, neutral titles), and added the two new figures to the maps PDF, which now has 23.

What I still cannot fix from my desk: I checked the processor version on sample days only, not on every file. My weather controls are country averages, not pixel level. I have no ground monitor validation. And I have no source yet for how closely UK nd Swiss vehicle standards follow the EU ones. These are written in the limitations section as they are.

Entry 37

Entry 36 ended with a list of things I could not fix from my desk. Some of them I could after all, and they went into the paper without an entry here, so this one catches the log up.

Ground stations. I pulled hourly NO2 from the European Environment Agency through the `airbase` package (`validate_eea_stations.py`). I used verified data, and unverified data only for months the verified files lack. 1,543 background sampling points in 33 countries had enough months. Station and satellite agree (monthly r = 0.68). The main model on station data gives an EU-27 decline of −4.0% that is not significant (p = 0.12), about the same size as the satellite at the same places (−4.5%). In December 2023 the stations in Denmark and Sweden fell by 16% and 7%, while the satellite values there went negative. So most of that month is a retrieval problem, not air quality. The United Kingdom has EEA data only for 2019, and Albania and Montenegro have no station with enough months, so this check is missing them.

Positive control. I added a known 2.5%, 5% and 10% cut to EU-27 NO2 from July 2021 and re-ran both models (`causal_inference_positive_control.py`). The matched pixel model finds about four fifths of each cut, and it would detect a uniform cut of about 2.7% with 80% power. So the near-zero matched estimate is not just a weak test.

Real GDP. With World Bank real GDP per capita in place of nominal GDP (`causal_inference_real_gdp.py`) the country estimate stays about the same size (−1.36) but is no longer significant (p = 0.072). The matched estimate does not move (−0.21).

Two numbers in Entry 36 have changed. The share of random splits significant at 5% is now 8.3% (83 of 999) with t(35) critical values, not 9.6%. And the vehicle-standards point now cites the EU regulations, the UK's retained version of Regulation 715/2007 and a Swiss Federal Council release on the 2017 rules, and the paper lists it as one of three possible explanations, not as a finding.

The paper is on EarthArXiv (10.31223/X5GB87) and the code and data are archived on Zenodo (10.5281/zenodo.21756660). `reproduce_paper.py` runs the scripts behind every table and figure in order.

Last, I had the whole project checked again from zero and corrected what it found. The paper now says the stations show a much smaller December 2023 fall instead of none, it no longer calls the winter-free blocks a drift that starts before the law, it says the stations include unverified data for missing months, it gives the Zenodo DOI in the data statement, and it quotes Sager and Singer exactly (a factor of three). The executive summary, dashboard and both portfolio pages now say, as the paper does, that the randomization p-value of 0.17 is on the coefficient and 0.046 on the t-statistic. The paper also now says how that p-value is computed: the share of the 999 assignments at least as extreme as the EU-27. Every copy now uses the same December 2023 wording. And the NO2 bar chart title still showed an old p-value (0.013); it now shows 0.018.